

PROJECT_CONVENTIONS

분실물 시스템 프로젝트 규칙

팀: 서진(백엔드/프론트/Dobot/키오스크), 시은(CV/AI/TurtleBot/매칭)

작성일: 2026-05-18

1. 깃허브 저장소 전략

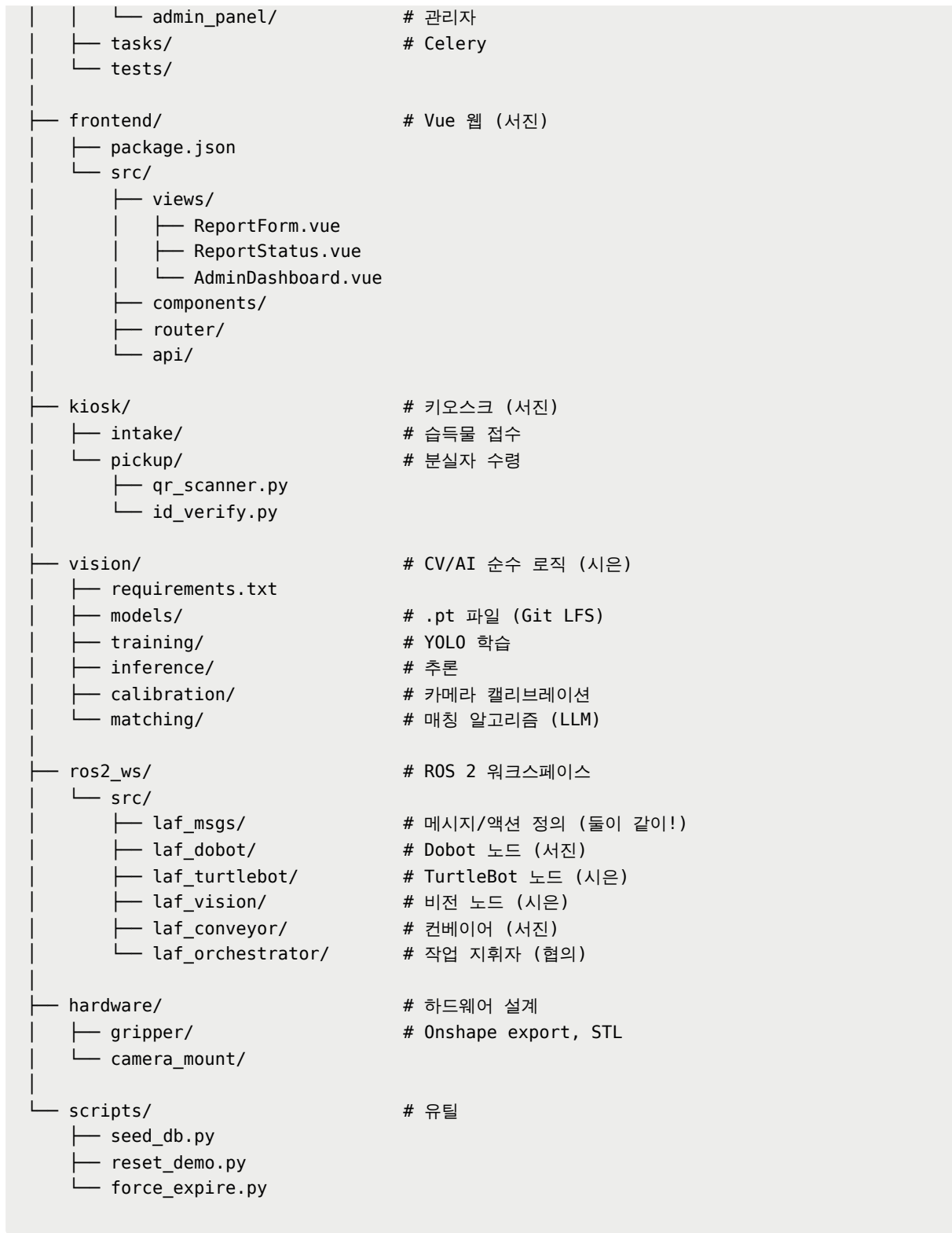
모노레포(monorepo)로 진행합니다.

- 저장소 1개에 모든 모듈을 같이 둠
- 백엔드 ↔ ROS ↔ AI 간 인터페이스를 한 곳에서 맞추기 쉬움
- 발표/시연 때 보여주기 편함
- 한 PR로 여러 모듈 동시 수정 가능

저장소 이름 예시: `lost-and-found` 또는 `laf-system`

2. 폴더 구조

```
lost-and-found/
├── README.md
├── .gitignore
├── .env.example
├──
├── docs/                                # 문서
│   ├── architecture.md
│   ├── api.md                          # REST API 명세
│   ├── ros_interface.md                # ROS Action/Topic 명세
│   ├── setup.md
│   └── demo-script.md
├──
├── backend/                             # Django (서진)
│   ├── manage.py
│   ├── requirements.txt
│   ├── config/                         # Django settings
│   ├── apps/
│   │   ├── items/                     # 분실물
│   │   ├── reports/                   # 신고
│   │   ├── warehouse/                 # 창고 슬롯
│   │   ├── matching/                  # 매칭
│   │   ├── pickup/                    # 수령/QR/본인확인
│   │   └── robot_tasks/                # 작업 큐
```



폴더 역할 요약

폴더	무엇이 들어가나
backend/	Django 서버, DB, REST API, Celery
frontend/	신고 폼, 신고 조회, 관리자 대시보드

폴더	무엇이 들어가나
<code>kiosk/</code>	접수/반환 키오스크 앱 (Python)
<code>vision/</code>	ROS와 무관한 순수 CV 로직 (학습 스크립트, 캘리브레이션)
<code>ros2_ws/</code>	모든 ROS 2 노드 (Dobot, TurtleBot, 비전 노드, Orchestrator)
<code>hardware/</code>	그리퍼, 마운트 등 물리 설계 파일

⚠ `vision/` 의 추론 로직은 ROS 노드(`ros2_ws/src/laf_vision/`)가 import해서 사용합니다. 학습/캘리브레이션 같은 일회성 작업은 `vision/` 에서 직접 실행.

3. 네이밍 규칙

파일/폴더

- Python 파일: `소문자_언더스코어` (`dobot_driver.py`)
- Vue 컴포넌트: `PascalCase.vue` (`ReportForm.vue`)
- ROS 패키지: `laf_xxx` (lost-and-found 약자)

Git 브랜치

- `main` — 항상 동작 상태 유지
- `dev` — 통합 브랜치
- `feature/짧은-설명` — 새 기능 (`feature/매칭-알고리즘`)
- `fix/짧은-설명` — 버그 수정 (`fix/dobot-피킹-실패`)

커밋 메시지

[모듈] 한글 설명 형식

예시:

- `[backend]` 신고 접수 API 추가
- `[ros]` Dobot pickup action 서버 구현
- `[vision]` YOLO 추론 노드 작성
- `[frontend]` 신고 폼 동의 체크박스 추가
- `[docs]` API 명세 업데이트

DB 테이블/컬럼

- 테이블명: 복수형 `snake_case` (`items` , `reports` , `warehouse_slots`)

- 컬럼명: snake_case (`created_at` , `slot_id` , `is_occupied`)

4. Git 워크플로우

1. `main` 에서 직접 작업 금지
2. `dev` 에서 본인 작업 브랜치 분기 (`feature/xxx`)
3. 작업 완료 후 `dev` 로 PR
4. 서로 리뷰 후 머지
5. 시연 직전에 `dev` → `main` 머지

`.gitignore` 필수 항목

```
# 환경변수
.env
*.env
!.env.example

# Python
__pycache__/
*.pyc
*.pyo
venv/
.venv/

# Node
node_modules/
dist/

# ROS 2
build/
install/
log/

# 큰 파일 (Git LFS 또는 별도 공유)
*.pt
*.pgm
dataset/
models/checkpoints/

# IDE
.vscode/
.idea/

# OS
.DS_Store
Thumbs.db
```

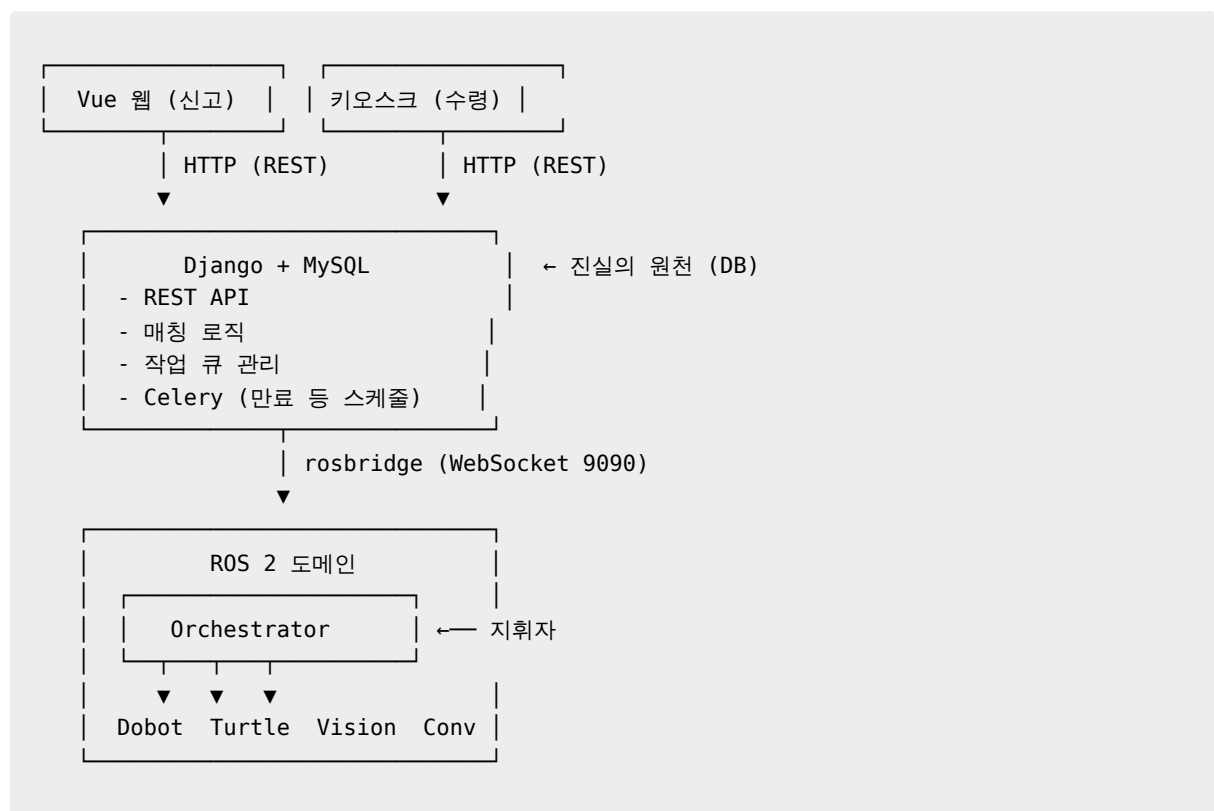
Git LFS 또는 별도 공유 (둘 중 선택)

- YOLO 모델 (`.pt`) — 보통 100MB+
- SLAM 맵 (`.pgm`)
- 데이터셋 사진

이런 큰 파일은 미리 정해두기:

- [] Git LFS 사용 / 또는
- [] 구글 드라이브 등 별도 공유 (경로만 README에 명시)

5. 시스템 아키텍처



핵심 원칙

1. Django만 DB를 만든다. ROS는 DB를 모름
2. ROS는 좌표/명령만 받는다. 신고자 정보, 매칭 점수 같은 비즈니스 데이터는 안 받음
3. 키오스크는 Django하고만 통신한다. ROS와 직접 통신 금지
4. 실시간 상태(로봇 위치 등)는 ROS 안에서만 흐른다. DB에 저장 X

6. 통신 규칙: 키오스크 ↔ Django ↔ ROS

(A) 키오스크 ↔ Django: REST API (HTTP/JSON)

키오스크는 그냥 웹 클라이언트처럼 Django에 HTTP 요청을 보냅니다.

주요 엔드포인트

접수 키오스크 (습득물 등록)

```
POST /api/items/
{
  "category": "wallet",
  "description": "검정 가죽 반지갑",
  "photo_path": "/uploads/abc.jpg",
  "found_at": "2026-05-18T14:30:00Z",
  "found_location": "강의동 1층"
}
→ 200 OK { "item_id": 42, "slot_id": "A-3" }
```

반환 키오스크 (분실자 수령)

```
POST /api/pickup/verify-code/
{
  "code": "8F3K2A" ← QR에서 읽은 코드
}
→ 200 OK { "report_id": 17, "item_id": 42, "item_photo": "..." }

POST /api/pickup/id-verify/
{
  "report_id": 17,
  "ocr_name": "김민수",
  "ocr_birth": "990315"
}
→ 200 OK { "matched": true }

POST /api/pickup/confirm/
{
  "report_id": 17,
  "decision": "accept" ← 또는 "reject"
}
→ 200 OK { "status": "robot_dispatched" }
```

응답 규칙

- 성공: 200 OK + JSON
- 클라이언트 오류 (잘못된 입력): 400 Bad Request
- 권한/검증 실패: 403 Forbidden

- 못 찾음: 404 Not Found
- 서버 오류: 500

응답 JSON 형식 통일:

```
// 성공
{ "status": "ok", "data": { ... } }

// 실패
{ "status": "error", "code": "INVALID_CODE", "message":
  "..."}

```

(B) Django ↔ ROS: rosbridge

Django는 `roslibpy` 로 rosbridge(WebSocket 9090)에 연결해서 ROS Action을 호출합니다.

방향 1: Django → ROS (명령)

작업 종류별 Action:

<code>/orchestrator/handle_pickup</code>	# 매칭 성공 → 분실자에게 전달
<code>/orchestrator/handle_return</code>	# 분실자 거부 → 다시 참고로
<code>/orchestrator/handle_discard</code>	# 만료 → 폐기함
<code>/orchestrator/handle_intake</code>	# 컨베이어 → 참고 적재

예시 Action goal (`HandlePickup.action`):

```
# Goal (Django → ROS)
int32 task_id          # DB의 robot_tasks.id
int32 item_id          # 어떤 물건인지 (식별용)
string slot_id         # "A-3" 같은 슬롯 ID
geometry_msgs/Pose slot_pose # 슬롯 실제 좌표
string destination     # "return_counter" 등 이름
---
# Result (ROS → Django, Action 완료 시)
bool success
string error_code      # 실패 시 "PICKUP_FAILED" 등
string final_state     # "delivered" 등
---
# Feedback (선택, 진행 중)
string current_stage   # "dobot_picking", "turtlebot_moving" 등
float32 progress       # 0.0 ~ 1.0

```

방향 2: ROS → Django (결과 콜백)

ROS Orchestrator가 작업 완료 시 Django REST API 호출:

```
POST /api/internal/robot-task-complete/
{
  "task_id": "uuid-...",
  "item_id": 42,
  "status": "success",    ← 또는 "failed"
  "error_code": null,     ← 실패 시 코드
  "final_state": "delivered"
}
```

⚠ `/api/internal/` 경로는 내부 통신용. 외부 인증 없이 같은 LAN에서만 접근 가능하게 설정.

보내지 않는 것

- ❌ 신고자 이름, 이메일, 신분증 정보
- ❌ DB 테이블 전체
- ❌ 매칭 점수, 카테고리 등 비즈니스 데이터
- ❌ 사진 원본 (필요하면 ROS가 별도로 가져감)

보내는 것

- ✅ 작업 식별자 (`task_id` , `item_id`)
- ✅ 좌표, 슬롯 ID, 목적지 이름
- ✅ 작업 타입 (pickup/return/discard/intake)

(C) ROS 내부: Topic / Action / Service

ROS 노드끼리는 표준 ROS 2 방식 사용. rosbridge 거치지 않음.

```
# Topic (실시간 상태 스트리밍, 휘발성)
/turtlebot/amcl_pose
/dobot/joint_states
/vision/item_detected

# Action (시간 걸리는 작업, 진행률 + 취소 가능)
/dobot/pickup
/turtlebot/navigate_to_pose
/orchestrator/handle_pickup

# Service (즉시 응답, 짧은 요청)
```



```
/conveyor/start  
/conveyor/stop
```

7. 인터페이스 정의 — 둘이 같이 먼저 합의할 것

프로젝트 초반에 이것 같이 정의해야 그 다음부터 각자 독립 작업 가능합니다.

합의 필요 항목

- ☐ `laf_msgs` 패키지에 들어갈 `.action`, `.msg` 파일 전부 정의
- ☐ REST API 엔드포인트 목록과 요청/응답 JSON 형식 (`docs/api.md`)
- ☐ DB 스키마 (테이블, 컬럼, 외래키)
- ☐ 좌표계 정의 (Dobot 좌표계, TurtleBot 맵 좌표계, 카메라 픽셀 좌표계)
- ☐ 에러 코드 목록 (`PICKUP_FAILED`, `OCR_MISMATCH`, `SLOT_FULL` 등)

8. 환경변수 (`.env`)

`.env.example` 을 만들어 두고 각자 `.env` 를 로컬에서 채워서 사용.

```
# Django  
DJANGO_SECRET_KEY=...  
DJANGO_DEBUG=True  
DATABASE_URL=mysql://user:pass@localhost/laf  
  
# Email (매칭 메일)  
SMTP_HOST=smtp.gmail.com  
SMTP_PORT=587  
SMTP_USER=...  
SMTP_PASSWORD=...  
  
# ROS  
ROSBRIDGE_HOST=localhost  
ROSBRIDGE_PORT=9090  
  
# Dobot  
DOBOT_IP=192.168.0.10
```

```
# 시연용
USE_SIM=true           # true: Gazebo/RoboDK, false: 실
기
DEMO_MODE=false        # true면 만료 기간 단축 등
EXPIRY_DAYS=30         # 시연 시 1로 변경
```

9. 작업 분담 정리

영역	담당	폴더/패키지
DB, Django REST API	서진	backend/
신고 웹, 관리자 대시보드	서진	frontend/
키오스크 (접수/반환)	서진	kiosk/
Dobot 제어 + ROS 노드	서진	ros2_ws/src/laf_dobot/
컨베이어 제어	서진	ros2_ws/src/laf_conveyor/
그리퍼 설계/제작	서진	hardware/gripper/
YOLO 학습 + 추론	시은	vision/ , ros2_ws/src/laf_vision/
카메라 캘리브레이션	시은	vision/calibration/
매칭 알고리즘 (LLM)	시은	vision/matching/
TurtleBot + Nav2 + SLAM	시은	ros2_ws/src/laf_turtlebot/
메시지/액션 정의	같이	ros2_ws/src/laf_msgs/
Orchestrator	협의	ros2_ws/src/laf_orchestrator/
데이터셋 수집	같이	vision/dataset/

10. 처음에 할 일 (순서대로)

- ☐ 깃허브 저장소 생성, 둘 다 collaborator 추가
- ☐ 이 문서를 docs/CONVENTIONS.md 로 커밋
- ☐ 위 폴더 구조 그대로 빈 폴더 + .gitkeep 커밋
- ☐ .env.example , .gitignore 작성
- ☐ DB 스키마 같이 그리고 docs/db_schema.md 로 저장
- ☐ REST API 엔드포인트 목록 같이 그리고 docs/api.md 로 저장
- ☐ laf_msgs 의 Action/Message 파일 같이 정의

☐ 각자 본인 영역 셋업 시작

부록: 자주 헷갈리는 것

Q. 키오스크가 ROS에 직접 명령하면 안 되나요?

A. 안 됩니다. 인증/검증 로직이 Django에 있고, DB 업데이트도 Django가 해야 해서요. 키오스크 → Django → ROS 순서.

Q. 로봇 현재 위치를 신고 조회 페이지에서 보여주고 싶으면?

A. Django가 rosbridge로 Topic 구독 → WebSocket으로 프론트에 전달. DB에는 저장 안 함.

Q. Vue에서 직접 rosbridge에 연결할 수 있나요?

A. 기술적으론 가능(`roslibjs`)하지만, 권한/보안상 Django를 거치는 게 안전합니다. 관리자 대시보드의 로봇 모니터링용으로만 제한적으로 쓸 수도 있음.

Q. 학습된 YOLO 모델은 어디에 두나요?

A. 파일 자체는 `vision/models/` 에 두되, Git LFS 또는 별도 공유. ROS 비전 노드는 환경변수로 경로 받아서 로드.